

Module : The convolution operation

Input

a	b	c	d
e	f	g	h
i	j	k	ℓ

Kernel

w	x
y	z

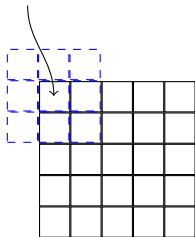
- Let us apply this idea to a toy example and see the results

Output

$aw+bx+ey+fz$	$bw+cx+fy+gz$	$cw+dx+gy+hz$
$ew+fx+iy+jz$	$fw+gx+jy+kz$	$gw+hx+ky+\ell z$

$$S_{ij} = (I * K)_{ij} = \sum_{a=\lfloor -\frac{m}{2} \rfloor}^{\lfloor \frac{m}{2} \rfloor} \sum_{b=\lfloor -\frac{n}{2} \rfloor}^{\lfloor \frac{n}{2} \rfloor} I_{i-a, j-b} K_{\frac{m}{2}+a, \frac{n}{2}+b}$$

pixel of interest



- For the rest of the discussion we will use the following formula for convolution
- In other words we will assume that the kernel is centered on the pixel of interest
- So we will be looking at both preceding and succeeding neighbors

Let us see some examples of 2D convolutions applied to images



$$\begin{matrix} & 1 & 1 & 1 \\ * & 1 & 1 & 1 \\ & 1 & 1 & 1 \end{matrix} =$$



blurs the image



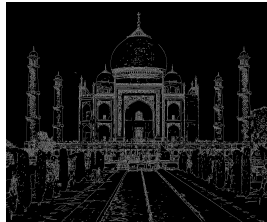
$$\begin{matrix} & 0 & -1 & 0 \\ * & -1 & 5 & -1 \\ & 0 & -1 & 0 \end{matrix} =$$



sharpens the image



$$\begin{matrix} & 1 & 1 & 1 \\ * & 1 & -8 & 1 \\ & 1 & 1 & 1 \end{matrix} =$$



detects the edges

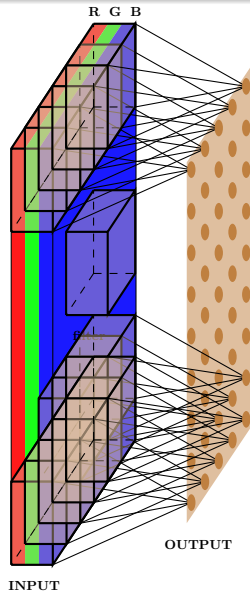
We will now see a working example of 2D convolution.

Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional output
- What would happen in the 3D case?

a	b	c	d
e	f	g	h
i	j	k	l

A	B	C	B	A	B	C
---	---	---	---	---	---	---



- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)
- As a result the output will be 2D (only width and height, no depth)
- Once again we can apply multiple filters to get multiple feature maps

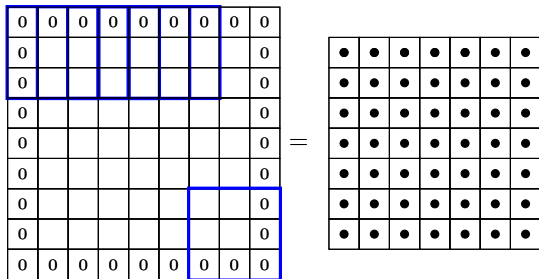
Relation between input size, output size and filter size

In general, $W_2 = W_1 - F + 1$

$$H_2 = H_1 - F + 1$$

We will refine this formula further

- Let us compute the dimension (W_2, H_2) of the output
- Notice that we can't place the kernel at the corners as it will cross the input boundary
- This is true for all the shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than the input
- As the size of the kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



- What if we want the output to be of same size as the input?
- We can use something known as padding
- Pad the inputs with appropriate number of 0 inputs so that you can now apply the kernel at the corners
- Let us use pad $P = 1$ with a 3×3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right

We now have,

$$W_2 = W_1 - F + 2P + 1$$

$$H_2 = H_1 - F + 2P + 1$$

We will refine this formula further

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

=

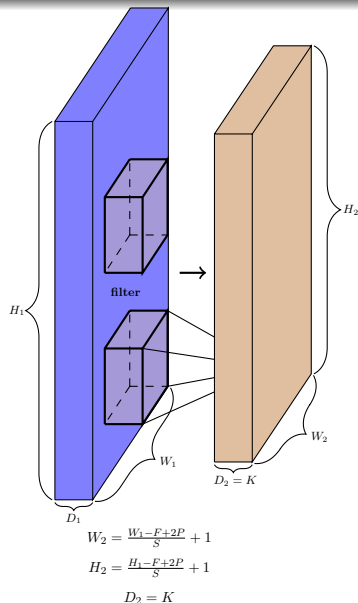
•	•	•	•
•	•	•	•
•	•	•	•
•	•	•	•

- What does the stride S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Here, we are essentially skipping every 2nd pixel which will again result in an output which is of smaller dimensions

So what should our final formula look like,

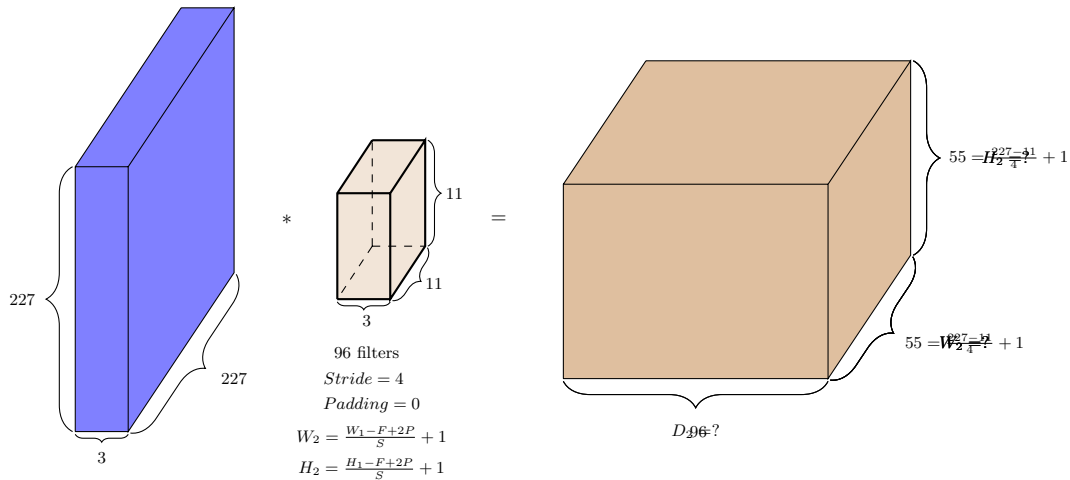
$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

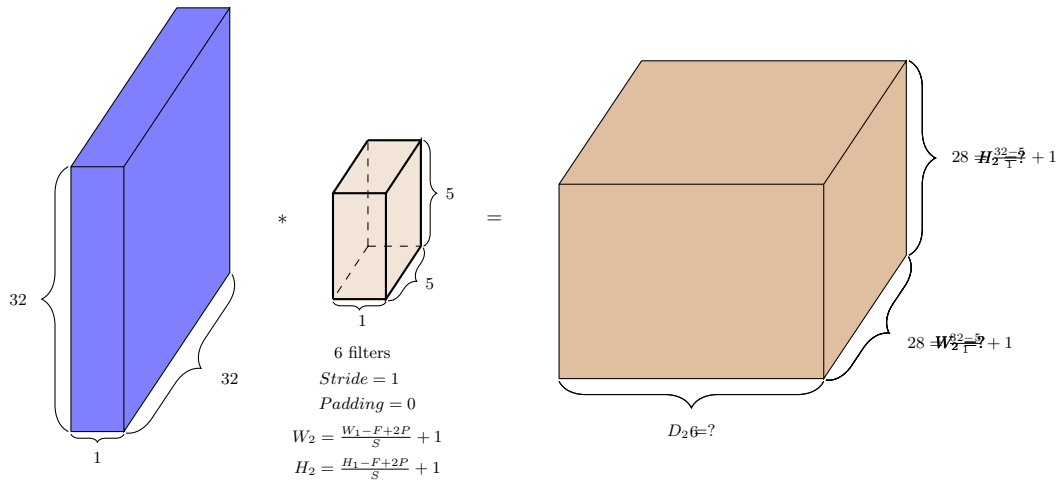


- Finally, coming to the depth of the output.
- Each filter gives us one 2D output.
- K filters will give us K such 2D outputs
- We can think of the resulting output as $K \times W_2 \times H_2$ volume
- Thus $D_2 = K$

Let us do a few exercises



Let us do a few exercises



Module: Convolutional Neural Networks

Features



Raw pixels



→ car, bus, **monument**, flower



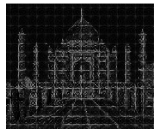
Edge Detector



→ car, bus, **monument**, flower



SIFT/HOG



→ car, bus, **monument**, flower

static feature extraction (no learning)

learning weights of classifier

Input

Features

Classifier



car, bus, monument, flower

0	0	0	0	0
0	1	1	1	0
0	1	-8	1	0
0	1	1	1	0
0	0	0	0	0



car, bus, monument, flower

-1.2135899e-03	3.2063686e-03	...	...	-2.0603730e-02
-1.3275782e-03	2.3633833e-03	...	...	-1.1924838e-02
...	...	...	...	...
...	...	...	...	...
-8.9522900e-04	-5.1489317e-03	...	...	-9.9086527e-03



Learn these weights

- Instead of using handcrafted kernels such as edge detectors **can we learn meaningful kernels/filters in addition to learning the weights of the classifier?**

Input



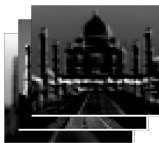
Features



Classifier

car, bus, monument, flower

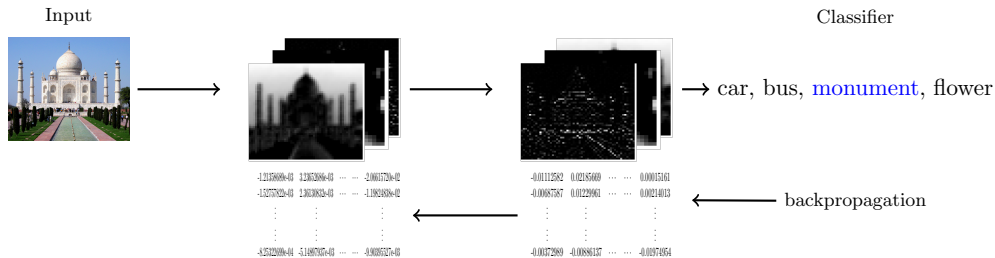
```
0 0 0 0 0
0 1 1 1 0
0 1 -8 1 0
0 1 1 1 0
0 0 0 0 0
```



car, bus, monument, flower

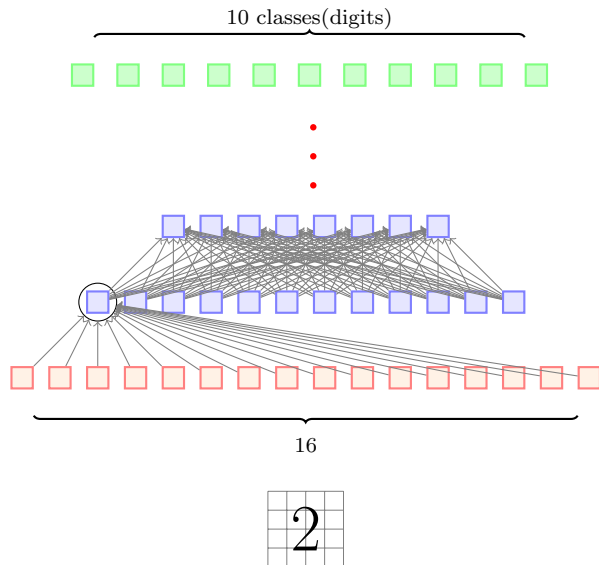
```
-0.0033776181331-0.020637994483 ..... -0.001667357820
-0.007719282540-0.000308539379 ..... -0.043297411091955402
:: :: ::
:: :: ::
-0.00309807704-0.00483020776 ..... -0.003707079286
```

- Even better: Instead of using handcrafted kernels (such as edge detectors) can we learn **multiple** meaningful kernels/filters in addition to learning the weights of the classifier?

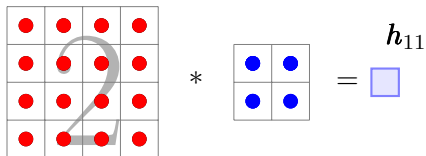
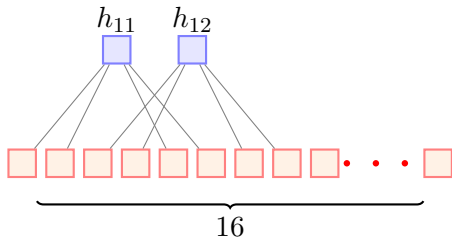


- Can we learn multiple **layers** of meaningful kernels/filters in addition to learning the weights of the classifier?
- Yes, we can !
- Simply by treating these kernels as parameters and learning them in addition to the weights of the classifier (using back propagation)
- Such a network is called a Convolutional Neural Network.

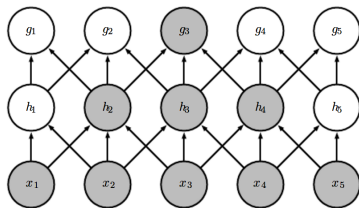
- Okay, I get it that the idea is to learn the kernel/filters by just treating them as parameters of the classification model
- But how is this different from a regular feedforward neural network
- Let us see



- This is what a regular feed-forward neural network will look like
- There are many dense connections here
- For example all the 16 input neurons are contributing to the computation of h_{11}
- Contrast this to what happens in the case of convolution

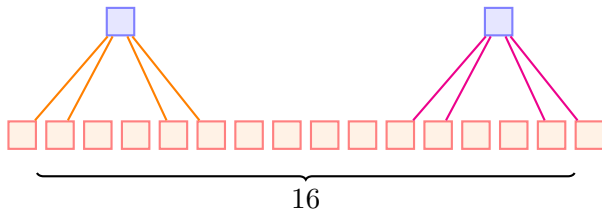


- Only a few local neurons participate in the computation of h_{11}
- For example, only pixels 1, 2, 5, 6 contribute to h_{11}
- The connections are much sparser
- We are taking advantage of the structure of the image (interactions between neighboring pixels are more interesting)
- This **sparse connectivity** reduces the number of parameters in the model



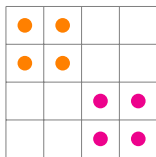
- But is sparse connectivity really good thing ?
- Aren't we losing information (by losing interactions between some input pixels)
- Well, not really
- The two highlighted neurons (x_1 & x_5)* do not interact in *layer 1*
- But they indirectly contribute to the computation of g_3 and hence interact indirectly

* Goodfellow-et-al-2016



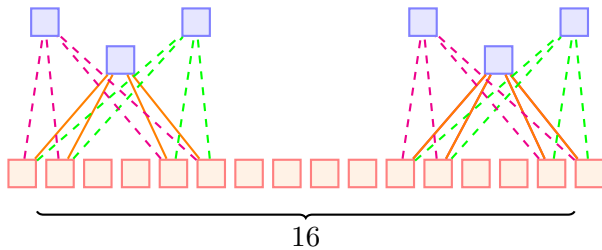
● Kernel 1

● Kernel 2

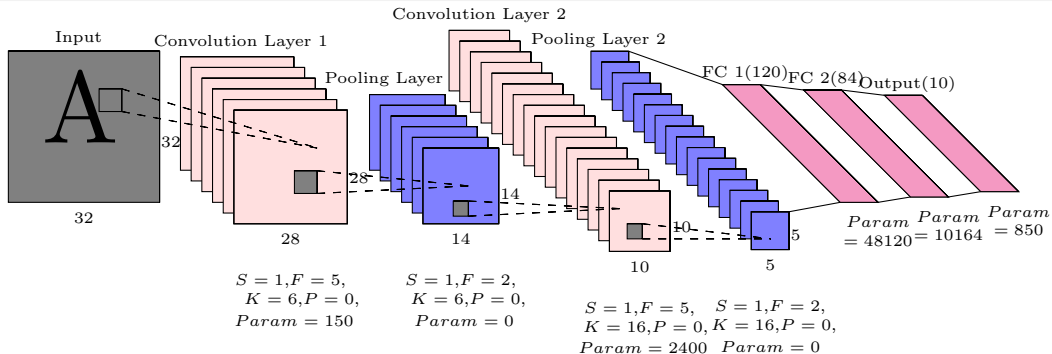


4x4 Image

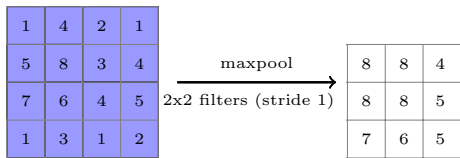
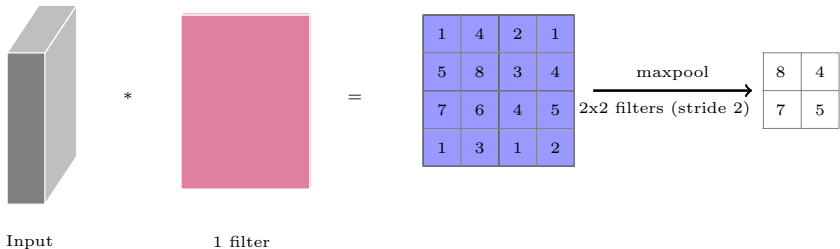
- Another characteristic of CNNs is **weight sharing**
- Consider the following network
- Do we want the kernel weights to be different for different portions of the image?
- Imagine that we are trying to learn a kernel that detects edges
- Shouldn't we be applying the same kernel at all the portions of the image?



- In other words shouldn't the *orange* and *pink* kernels be the same
- Yes, indeed
- This would make the job of learning easier (instead of trying to learn the same weights/kernels at different locations again and again)
- But does that mean we can have only one kernel?
- No, we can have many such kernels but the kernels will be shared by all locations in the image
- This is called “weight sharing”



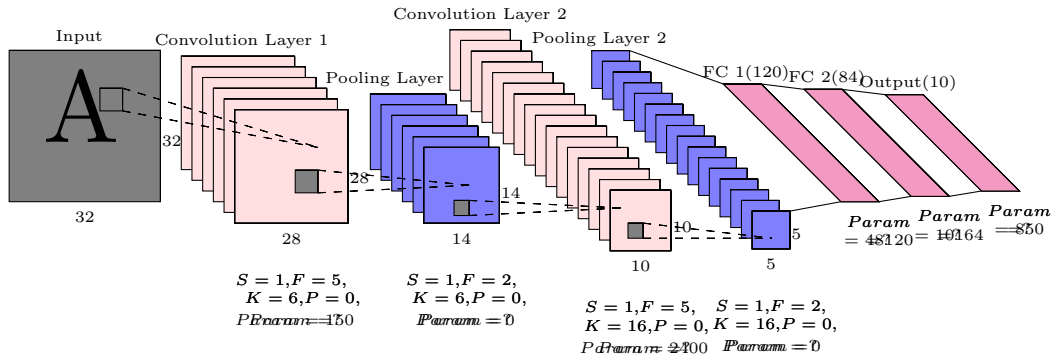
- It has alternate convolution and pooling layers
- What does a pooling layer do?
- Let us see



- Instead of max pooling we can also do average pooling

We will now see some case studies where convolution neural networks have been successful

LeNet-5 for handwritten character recognition



- How do we train a convolutional neural network ?

Input

b	c	d
e	f	g
h	i	j

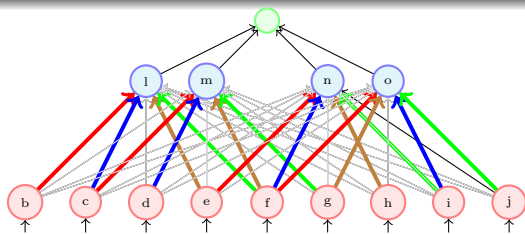
Kernel

w	x
y	z

Output

ℓ	m
n	o

- We can thus train a convolution neural network using backpropagation by thinking of it as a feedforward neural network with sparse connections

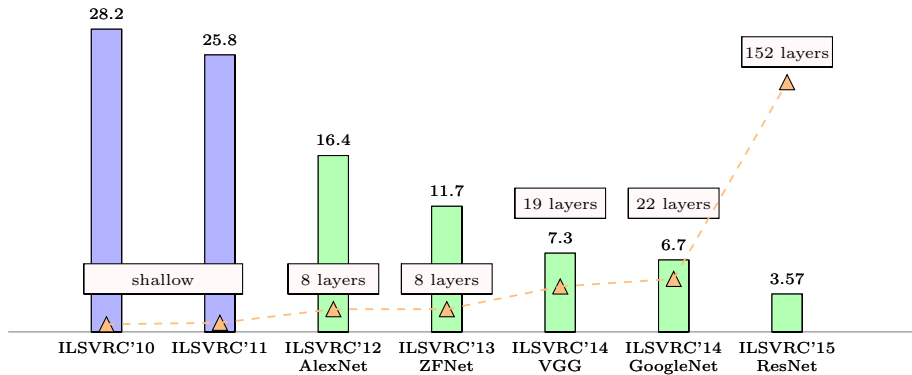


- A CNN can be implemented as a feedforward neural network
- wherein only a few weights (in color) are active
- the rest of the weights (in gray) are zero

Module : CNNs (success stories on ImageNet)

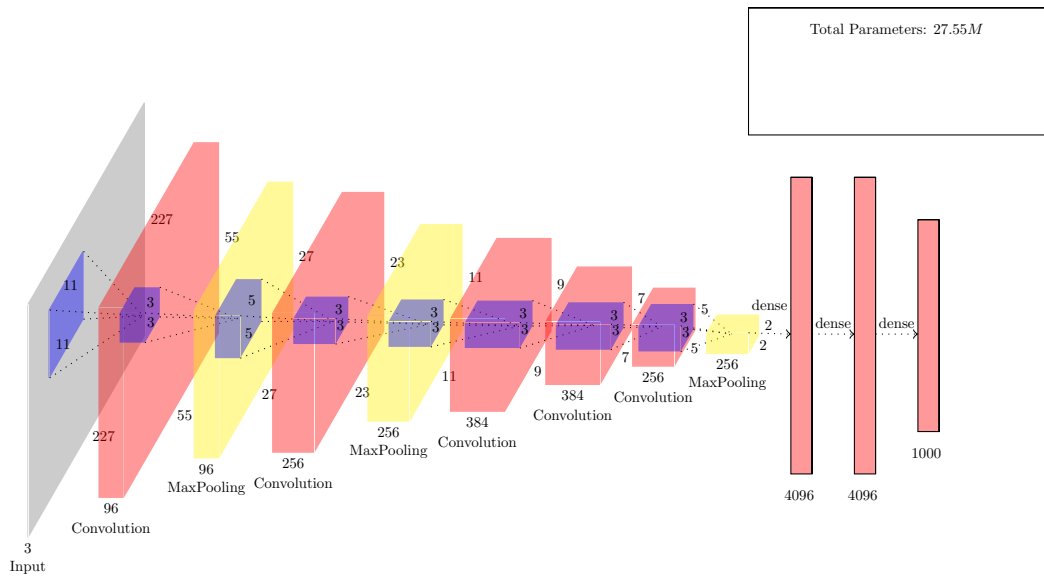
ImageNet Success Stories(roadmap for rest of the talk)

- AlexNet
- VGGNet
-

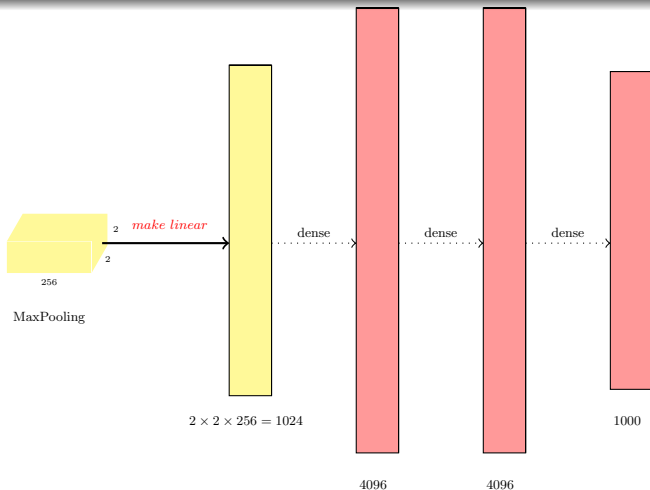


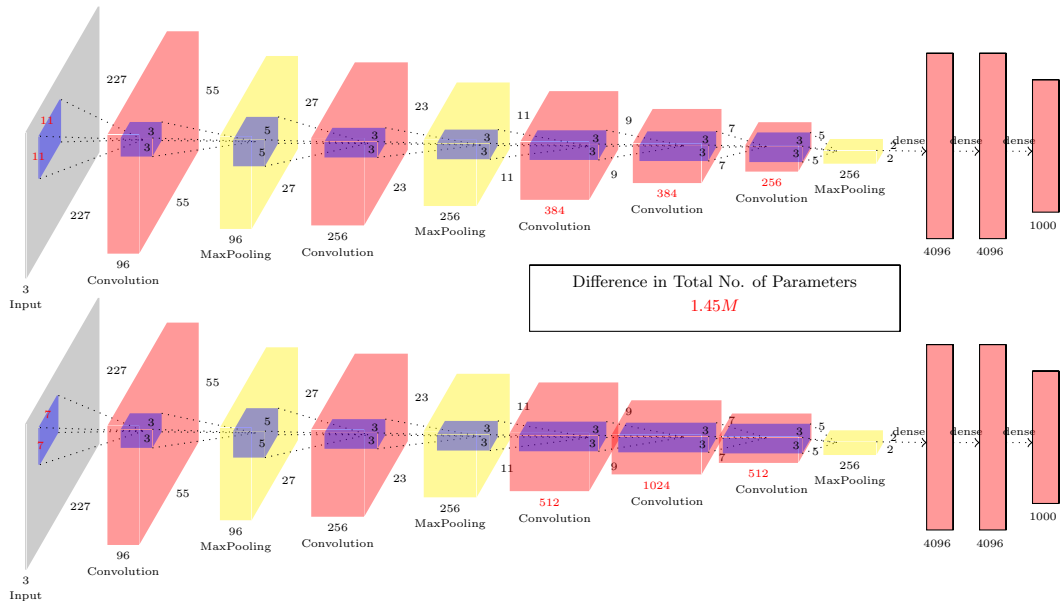
ImageNet Success Stories(roadmap for rest of the talk)

- AlexNet
- VGGNet
-



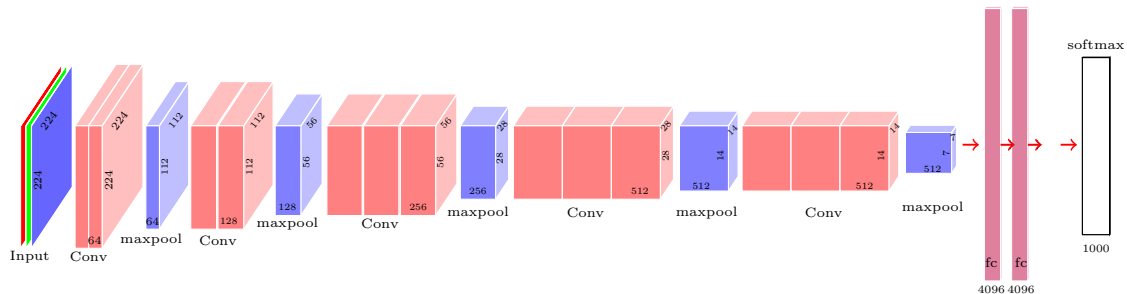
- Let us look at the connections in the fully connected layers in more detail
- We will first stretch out the last conv or maxpool layer to make it a 1d vector
- This 1d vector is then densely connected to other layers just as in a regular feedforward neural network





ImageNet Success Stories(roadmap for rest of the talk)

- AlexNet
- VGGNet
-



- Kernel size is 3×3 throughout
- Total parameters in non FC layers = $\sim 16M$
- Total Parameters in FC layers = $(512 \times 7 \times 7 \times 4096) + (4096 \times 4096) + (4096 \times 1024) = \sim 122M$
- Most parameters are in the first FC layer ($\sim 102M$)